AI Support System Architecture

Zara Nails Multi-Tenant Platform

Document version: 1.0
Date: April 2026
Author: Architecture session with GitHub Copilot
Related documents:

- `MULTITENANT_MASTER_INDEX.md` — program overview and week-by-week roadmap
- `MULTITENANT_COMPANION_EXECUTION_BLUEPRINT.md` — Firestore schema v1, 20-week roadmap
- `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md` — public landing shell and discovery scaffold prerequisites
- `MULTITENANT_WEEKS_5_TO_8_COPILOT_PROMPTS.md` — build prompts for Weeks 5–8 (support skeleton)
- `MULTITENANT_WEEKS_9_TO_12_COPILOT_PROMPTS.md` — build prompts for Weeks 9–12 (AI layer)

Table of Contents

1. System Overview
2. Design Decisions & Rationale
3. Architecture Diagram
4. Component Breakdown
5. Firestore Schema
6. AI Routing Logic
7. Escalation Rules
8. Knowledge Base Specification
9. Security Model
10. Implementation Phases & Roadmap Cross-Reference
11. Copilot Prompt Blocks
12. Open Decisions Register

---

1. System Overview

The AI Support System is a **fully automated first-response layer** embedded inside the Zara Nails mobile app. It handles support requests from two user classes:

- **Tenant admins** (salon owners and their staff) — questions about features, setup, billing, account management
- **End clients** (customers of those salons) — questions about bookings, login, loyalty points, app usage

You (the platform owner) are the only human escalation target. The system is designed so that common and solvable issues never reach you. When they do, you have full context and a draft reply ready.

Design principles:

- AI is a backend service, not embedded app logic — upgradeable without an app release
- Escalation is explicit and contextual — not a fallback for AI errors
- You maintain a plain-text knowledge base — no code changes required to update it
- Tenant isolation is enforced — no cross-tenant data leakage in support context

---

2. Design Decisions & Rationale

Decision	Choice	Reason
Support channel	In-app chat only	Single surface, no email thread management; consistent UX
AI provider	OpenAI GPT-4o	Best structured output + function-calling for routing decisions; cost trivial at support volumes
Integration point	Firebase Cloud Function	No new infrastructure; reuses existing Firebase project; hot-swappable
Escalation delivery	Email + admin queue	Email for immediate awareness; in-app queue for full context and reply drafting
Knowledge base	System prompt (plain text)	Simple to maintain by a non-developer; good enough until volume justifies RAG
Response mode	AI sends directly; escalates when not confident	Minimises your involvement; AI confidence score drives the decision
Context injection	User role + recent bookings/activity	Increases first-response accuracy without exposing other tenant data

---

3. Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│  Mobile App (React Native)                                       │
│                                                                  │
│  ┌──────────────────────┐    ┌──────────────────────────────┐   │
│  │  SupportChatScreen   │    │  AdminSupportQueueScreen     │   │
│  │  (client + admin)    │    │  (you, on your device)       │   │
│  └──────────────────────┘    └──────────────────────────────┘   │
│           │ writes/reads                  │ reads + replies     │
└───────────┼───────────────────────────────┼─────────────────────┘
            │                               │
            ▼                               ▼
┌──────────────────────────────────────────────────────────────────┐
│  Firestore                                                       │
│                                                                  │
│  tenants/{tenantId}/                                             │
│    supportTickets/{ticketId}          ← ticket document          │
│      messages/{messageId}            ← chat thread               │
└──────────────────────┬───────────────────────────────────────────┘
                       │
```

```
                          │ onCreate trigger
                          ▼
┌──────────────────────────────────────────────────────────────────┐
│  Cloud Function: handleSupportMessage                            │
│                                                                  │
│  1. Load user context (role, recent bookings, last error)        │
│  2. Load knowledge base system prompt                            │
│  3. Call OpenAI GPT-4o with structured output                    │
│  4. Parse response: { answer, confidence, escalate, reason }     │
│                                                                  │
│  if confidence >= 0.75 and escalate == false:                    │
│    → write AI answer to messages subcollection                   │
│    → mark ticket status = "resolved"                             │
│                                                                  │
│  else:                                                           │
│    → write AI draft to ticket.draftReply                         │
│    → mark ticket status = "escalated"                            │
│    → call sendEscalationEmail()                                  │
│    → write to escalationQueue collection                         │
└──────────────────────────────────────────────────────────────────┘
                          │
                ┌─────────┴──────────┐
                ▼                    ▼
      ┌────────────────┐     ┌──────────────────────┐
      │  SendGrid /    │     │  escalationQueue     │
      │  Nodemailer    │     │  (admin app queue)   │
      │  email to you  │     │  visible in          │
      └────────────────┘     │  AdminSupportQueue   │
                             │  Screen              │
                             └──────────────────────┘
```

---

4. Component Breakdown

4.1 SupportChatScreen (mobile app)

- Available to all authenticated users (client and admin roles)
- Shows message thread for the current active ticket
- "New request" button opens a fresh ticket
- Displays AI badge on AI-generated responses
- Shows "Handed to support team" status when escalated, with expected response time message
- No live user-to-user chat — this is single-user → system only

4.2 AdminSupportQueueScreen (mobile app — your view)

- Lists all escalated tickets across all tenants, sorted by createdAt desc
- Each ticket shows: tenant name, user name, user role, message, AI draft reply, context snapshot
- You tap a ticket → see full message history + AI reasoning note
- Reply field is pre-populated with AI draft
- Actions: Send Reply, Dismiss (mark resolved without reply), Re-route to AI (retry with updated knowledge base)

- Reply sends a message back into the user's chat thread and marks ticket resolved

4.3 handleSupportMessage Cloud Function

- Triggered by Firestore `onCreate` on `tenants/{tenantId}/supportTickets/{ticketId}/messages/{messageId}`
- Only triggers on messages where `role == "user"` (skips AI and admin replies)
- Loads context: user document, last 5 bookings, tenant settings
- Calls OpenAI with the assembled prompt
- Writes structured result back to Firestore
- Calls email escalation function if needed

4.4 Knowledge Base (system prompt document)

- Stored in Firestore: `platform/config` document, field `supportSystemPrompt` (string)
- You edit it via a simple admin UI field or directly in Firebase Console
- No redeploy needed to update — function reads it fresh on each invocation
- Structure described in Section 8

4.5 Escalation Email

- Reuses existing nodemailer/SendGrid setup from `functions/index.js`
- Sends to your registered platform owner email
- Subject: `[Support Escalated] {tenantName} — {userRole}: {first 60 chars of message}`
- Body: full message, user context, AI confidence score, AI draft reply, deep link to admin queue

4.6 Route Placement (Public vs Protected)

- `SupportChatScreen` is a **protected route** and requires authenticated tenant membership
- `AdminSupportQueueScreen` is a **protected route** and additionally requires platform owner role
- Public routes (`Landing`, `Login`, `Register`, `DiscoverBusinesses`) never expose support tickets or escalation data
- This route split is implemented before support build starts via:
  - Week 1 Task 1.5 in `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md`
  - Week 2 Task 2.5 in `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md`

---

5. Firestore Schema

The support system adds the following to the multi-tenant schema defined in `MULTITENANT_COMPANION_EXECUTION_BLUEPRINT.md`.

5.1 supportTickets (per-tenant subcollection)

```
tenants/{tenantId}/supportTickets/{ticketId}

{
  ticketId:        string,           // auto-id
  userId:          string,           // UID of requesting user
```

```
  userRole:        "client" | "admin" | "owner",
  userDisplayName: string,
  tenantId:        string,           // denormalised for admin queue queries
  tenantName:      string,           // denormalised
  status:          "open" | "ai_processing" | "resolved" | "escalated" | "closed",
  createdAt:       Timestamp,
  updatedAt:       Timestamp,
  resolvedAt:      Timestamp | null,
  draftReply:      string | null,    // AI-generated draft, shown in admin queue
  aiConfidence:    number | null,    // 0–1 score from last AI call
  aiReason:        string | null,    // AI's short reason for decision
  contextSnapshot: {                 // captured at ticket creation
    recentBookingIds: string[],
    lastBookingStatus: string | null,
    accountCreatedAt: Timestamp,
    locale: string
  }
}
```

5.2 messages (subcollection)

```
tenants/{tenantId}/supportTickets/{ticketId}/messages/{messageId}

{
  messageId:  string,
  role:       "user" | "ai" | "admin",
  content:    string,
  createdAt:  Timestamp,
  senderId:   string | null  // null for AI messages
}
```

5.3 escalationQueue (platform-level collection)

```
escalationQueue/{ticketId}

{
  ticketId:    string,
  tenantId:    string,
  tenantName:  string,
  userId:      string,
  userRole:    string,
  message:     string,       // first user message
  draftReply:  string,       // AI draft
  aiConfidence: number,
  createdAt:   Timestamp,
  status:      "pending" | "replied" | "dismissed"
}
```

This collection is **platform-level** (not per-tenant) so your AdminSupportQueueScreen can query across all tenants in a single read without cross-tenant data access by regular users. Firestore rules lock it to platform owner only.

5.4 platform/config (extended)

```
platform/config

{
  // ... existing fields ...
  supportSystemPrompt: string,       // knowledge base text, editable by owner
  supportEscalationEmail: string,    // your email
  supportEscalationThreshold: number // default 0.75; escalate if confidence below
this
}
```

---

6. AI Routing Logic

The Cloud Function calls GPT-4o with a **structured output schema** (`response_format: json_schema`) to get a deterministic, parseable routing decision.

6.1 Prompt structure

```
SYSTEM:
{supportSystemPrompt from platform/config}

---
You are the support AI for a nail salon booking platform.
The user is a {userRole} at tenant "{tenantName}".
Their account was created on {accountCreatedAt}.
Recent booking statuses: {recentBookingStatuses}.
Current locale: {locale}.

Respond with JSON matching this schema:
{
  "answer":     string,   // your full reply to the user (markdown allowed)
  "confidence": number,   // 0.0–1.0: how confident you are this answer is correct
  "escalate":   boolean,  // true if this should go to a human
  "reason":     string    // one sentence: why you are / are not escalating
}

Rules:
- Always set escalate: true for billing, account deletion, legal, abuse, or data
requests
- Always set escalate: true if confidence < {escalationThreshold}
- Never reveal tenant data from other tenants
- Never invent feature functionality that is not in your knowledge base
- If you don't know, say so and set escalate: true
```

```
USER:
{userMessage}
```

6.2 Routing decision tree

```
AI response received
        │
        ├─ escalate == true  ──────────────────────────────────────┐
        │                                                          │
        └─ escalate == false                                       │
                │                                                  │
                ├─ confidence >= threshold (default 0.75)          │
                │     → write answer as AI message                 │
                │     → ticket.status = "resolved"                 │
                │                                                  │
                └─ confidence < threshold  ────────────────────────┘
                                                                   │
                                                          ticket.status =
"escalated"
                                                          write draftReply
                                                          write to escalationQueue
                                                          send escalation email
```

---

7. Escalation Rules

Always escalate (regardless of confidence)

Category	Examples
Billing and payments	"Why was I charged?", "I want a refund", "Cancel my subscription"
Account deletion	"Delete my account and all my data"
Legal / GDPR	"Send me all my data", "I want to be forgotten"
Abuse reports	"This salon is scamming me", "I'm being harassed"
Authentication failure (persistent)	Login not working after standard troubleshooting
Multi-message unresolved	Ticket has >3 user messages without resolution

Escalate if confidence < threshold

Default threshold: **0.75** (configurable in `platform/config.supportEscalationThreshold`).

You can tune this over time. Start at 0.75 (lean slightly toward AI handling) and watch escalation rate. If you're getting too many trivial escalations, raise to 0.85. If AI mistakes are reaching users, lower to 0.65.

Auto-close without reply

If a ticket has status `"resolved"` and no further user message within 48 hours, a scheduled function marks it `"closed"`. This keeps the queue clean.

---

8. Knowledge Base Specification

The knowledge base is the plain-text `supportSystemPrompt` field in `platform/config`. You write and maintain it — no deployment needed.

Recommended structure

```
## PLATFORM OVERVIEW
[2-3 sentences describing what the platform does]

## FEATURES AVAILABLE TO SALON OWNERS (admin/owner role)
- Booking management: [brief description]
- Client management: [brief description]
- Staff scheduling: [brief description]
- Loyalty programme: [brief description]
- Messaging: [brief description]
- Waiting list: [brief description]
- Reports: [brief description]
- Gallery: [brief description]
- Services: [brief description]

## FEATURES AVAILABLE TO CLIENTS
- Booking: [brief description]
- Appointments view: [brief description]
- Loyalty points: [brief description]
- Messaging salon: [brief description]
- Waiting list: [brief description]

## COMMON ISSUES AND SOLUTIONS

### Can't log in
1. Check email address is correct
2. Use "Forgot Password" on login screen
3. If account was recently created, check approval status with salon admin

### Booking not appearing
1. Wait 30 seconds and refresh the screen
2. Check Appointments tab, not Home screen
3. Confirm booking was completed (you should have received a confirmation)

### Loyalty points incorrect
1. Points are added after the appointment is marked complete by the salon
2. Contact your salon admin if points seem wrong after a completed visit

### [Add more as you learn from real tickets]

## WHAT THIS AI CANNOT HELP WITH
- Billing, payments, and subscription changes → always escalated to platform
```

```
support team
- Account deletion and data export requests → always escalated
- Any issue that has been reported three times without resolution → escalated

## RESPONSE TONE
Friendly, concise, professional. Use bullet points for steps. Never use technical
jargon.
Address the user by first name if available.
```

How to update it

1. Open Firebase Console → Firestore → `platform` collection → `config` document
2. Edit the `supportSystemPrompt` field directly
3. Save — next support request automatically uses the updated text

No code change. No redeployment.

---

9. Security Model

Firestore rules additions

```
// Support tickets — user can only read/write their own tenant's tickets
match /tenants/{tenantId}/supportTickets/{ticketId} {
  allow read: if isAuthenticatedTenantMember(tenantId);
  allow create: if isAuthenticatedTenantMember(tenantId)
                && request.resource.data.userId == request.auth.uid;
  allow update: if false; // only Cloud Functions update tickets
}

match /tenants/{tenantId}/supportTickets/{ticketId}/messages/{messageId} {
  allow read: if isAuthenticatedTenantMember(tenantId);
  allow create: if isAuthenticatedTenantMember(tenantId)
                && request.resource.data.role == "user"
                && request.resource.data.senderId == request.auth.uid;
  allow update, delete: if false; // immutable
}

// Escalation queue — platform owner only
match /escalationQueue/{ticketId} {
  allow read, write: if isPlatformOwner();
}

// Platform config — owner reads; no client access
match /platform/config {
  allow read: if isPlatformOwner();
  allow write: if isPlatformOwner();
}
```

Context isolation

The Cloud Function only loads context for the `userId` and `tenantId` from the triggering message document. It never queries across tenants. The AI prompt contains only that single user's context — no other tenant's data is ever included.

OpenAI API key

- Stored in Firebase Functions config (`functions.config().openai.key`) or Secret Manager
- Never exposed to the client app
- Rotated if compromised — key rotation does not require app update

Subscription context (for feature gating, added in v1.1)

When AI support field tickets, Cloud Function will also load:

- Current tenant subscription tier (from tenantUsers.subscription.tier)
- Current subscription status (active, past_due, suspended, etc.)

This context allows AI to:

- Recommend tier upgrades based on requested features ("This feature is available in Professional tier")
- Acknowledge billing issues for past_due/suspended accounts ("We see your account is currently suspended due to payment")
- Route billing-related escalations to appropriate queue

Importantly:

- Subscription context is included in AI system prompt **only for information**
- Subscription gating for actual feature access is enforced in the app layer (Week 14+), not by support system
- Support system never blocks communication based on subscription status

---

10. Implementation Phases & Roadmap Cross-Reference

The support system is built in **three phases** across the expanded 20-week roadmap.

Prerequisites in Weeks 1-2:

- Week 1 Task 1.5 establishes Public vs Protected route groups and landing-first navigation.
- Week 2 Task 2.5 adds public discovery scaffolding.
- Support implementation then starts in Weeks 5-6 and remains in protected routes only.

Phase 1 — Skeleton (Weeks 5–6)

Goal: Get the support chat UI running with manual escalation only. No AI yet. Start collecting real user messages.

What to build:

- ☐ `SupportChatScreen` UI — message input, thread display, status badge
- ☐ `supportTickets` and `messages` Firestore collections with security rules

- ☐ `handleSupportMessage` Cloud Function — stub that auto-escalates everything (no AI call yet)
- ☐ `escalationQueue` collection
- ☐ Escalation email (simple, no draft reply yet)
- ☐ `AdminSupportQueueScreen` — basic list view, tap to view thread, manual reply field

Where this fits in the main roadmap:
→ Add as **Task 6.5** in `MULTITENANT_WEEKS_5_TO_8_COPILOT_PROMPTS.md` (Week 6, after Task 6.4 — Notification Preferences).
→ The messaging pattern has already been built in Task 7.x (Week 7) — reuse the same message thread UI component. → Navigation prerequisite comes from Week 1 Task 1.5 (`Public` vs `Protected` groups) in `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md`.

Value at this point: Every support request reaches you, but through a structured queue with full context — better than users emailing you directly with no context.

Phase 2 — AI Layer (Weeks 9–10)

Goal: Activate the AI routing. By now you have real escalated tickets as training examples. Update the knowledge base with what you learned.

What to build:

- ☐ OpenAI API integration in `handleSupportMessage` (replace auto-escalate stub)
- ☐ Structured output parsing and routing decision logic
- ☐ AI confidence score storage on ticket
- ☐ `draftReply` written to escalated tickets
- ☐ Admin queue updated to show draft reply pre-populated in reply field
- ☐ `platform/config.supportSystemPrompt` field — initial knowledge base text
- ☐ `supportEscalationThreshold` config field

Where this fits in the main roadmap:
→ Add as **Task 9.5** in `MULTITENANT_WEEKS_9_TO_12_COPILOT_PROMPTS.md` (Week 9, after Task 9.4 — Rating Aggregation).
→ This is the right time because you'll have real ticket data from the pilot to populate the knowledge base before AI goes live.

Value at this point: Most routine questions auto-resolved without you. You only see complex or sensitive issues with a draft reply ready.

Phase 3 — Polish and Tuning (Week 11–12)

Goal: Tune confidence threshold based on real data. Add admin tooling. Harden security.

What to build:

- ☐ Admin UI field to edit `supportSystemPrompt` (instead of Firebase Console)
- ☐ Admin UI slider/input for `supportEscalationThreshold`
- ☐ Auto-close scheduled function for resolved tickets (48h no reply → closed)
- ☐ Support ticket analytics: resolution rate, avg confidence, escalation rate, avg response time

- ☐ "Re-route to AI" button on escalated tickets (retry with updated knowledge base)
- ☐ AI badge on chat messages so users know they're talking to AI
- ☐ Rate limiting: max 10 open tickets per user to prevent abuse

Where this fits in the main roadmap:
→ Add as **Task 11.5** and **Task 12.5** in `MULTITENANT_WEEKS_9_TO_12_COPILOT_PROMPTS.md`.
→ Week 11 is the analytics week — support analytics fits naturally.
→ Week 12 is hardening — rate limiting and security review fit naturally.

Phase summary table

Phase	Weeks	Prompt file to open	Add after task
Prereq — Route foundation	1-2	`MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md`	Task 1.4 (then Task 2.4)
1 — Skeleton (no AI)	5–6	`MULTITENANT_WEEKS_5_TO_8_COPILOT_PROMPTS.md`	Task 6.4
2 — AI routing	9–10	`MULTITENANT_WEEKS_9_TO_12_COPILOT_PROMPTS.md`	Task 9.4
3 — Polish + tuning	11–12	`MULTITENANT_WEEKS_9_TO_12_COPILOT_PROMPTS.md`	Task 11.4 / 12.x

---

11. Copilot Prompt Blocks

Use these in VS Code with GitHub Copilot Agent mode in the new multi-tenant repository. Apply the Global Guardrails Prompt from `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md` before running any block below.

SUPPORT TASK 1 — Firestore Schema and Security Rules (run in Week 6)

```
Using the schema defined below, create the Firestore collections and security rule
additions for the AI support system.

Collections to create:
1. tenants/{tenantId}/supportTickets/{ticketId} — fields: ticketId, userId,
userRole, userDisplayName, tenantId, tenantName, status (enum:
open|ai_processing|resolved|escalated|closed), createdAt, updatedAt, resolvedAt,
draftReply, aiConfidence, aiReason, contextSnapshot { recentBookingIds,
lastBookingStatus, accountCreatedAt, locale }
2. tenants/{tenantId}/supportTickets/{ticketId}/messages/{messageId} — fields:
messageId, role (enum: user|ai|admin), content, createdAt, senderId
3. escalationQueue/{ticketId} — fields: ticketId, tenantId, tenantName, userId,
userRole, message, draftReply, aiConfidence, createdAt, status (enum:
pending|replied|dismissed)

Security rules:
- tenants/{tenantId}/supportTickets — authenticated tenant member can read; can
```

```
create only their own (userId == auth.uid); update only by Cloud Functions
- messages subcollection — member can read; can create only user-role messages
with own senderId; no update/delete
- escalationQueue — read and write only by isPlatformOwner()
- platform/config — read and write only by isPlatformOwner()

Use TypeScript interfaces in src/types/support.ts for all document shapes.
Write Firestore rules additions into firestore.rules using the existing helper
functions (isAuthenticatedTenantMember, isPlatformOwner).
Do not change any unrelated rules.
```

SUPPORT TASK 2 — SupportChatScreen UI (run in Week 6)

```
Create src/screens/shared/SupportChatScreen.tsx.

Requirements:
- Works for both client and admin/owner roles
- On mount: query tenants/{tenantId}/supportTickets where userId ==
currentUser.uid, status != "closed", orderBy createdAt desc, limit 1 — if none
exists, auto-create a new open ticket
- Subscribe to messages subcollection with onSnapshot for real-time updates
- Render message thread: user messages right-aligned, AI messages left-aligned
with small "AI" badge, admin messages left-aligned with "Support Team" label
- Show ticket status badge at top: "Open", "In Progress", "Resolved", "Waiting for
Support Team"
- When status == "escalated" show a notice: "Your request has been passed to our
support team. We'll respond within 24 hours."
- Message input at bottom with Send button; on send, write a new message document
(role: "user") to the messages subcollection
- New ticket button in header — creates a fresh ticket document, clears the thread
- Use the existing app design tokens and component patterns (same styles as
MessagesScreen)
- No direct AI calls from the client — all AI logic is in the Cloud Function
```

SUPPORT TASK 3 — handleSupportMessage Cloud Function Stub (run in Week 6)

```
Create a Cloud Function handleSupportMessage in functions/src/support.ts.

This is Phase 1 — no AI yet. Auto-escalate everything so we start collecting real
tickets.

Trigger: Firestore onCreate on
tenants/{tenantId}/supportTickets/{ticketId}/messages/{messageId}

Logic:
1. Return immediately if message.role != "user" (prevent loops)
2. Update parent ticket status to "ai_processing"
```

```
3. Load user document from tenants/{tenantId}/tenantUsers/{userId}
4. Load last 5 bookings from tenants/{tenantId}/bookings where customerId ==
userId, orderBy createdAt desc
5. Build contextSnapshot and update ticket document
6. Auto-escalate: set ticket status to "escalated", draftReply to null,
aiConfidence to null, aiReason to "Phase 1: manual review only"
7. Write mirror document to escalationQueue/{ticketId}
8. Call sendEscalationEmail(ticket, user, tenant) — sends email to platform owner

sendEscalationEmail:
- Use existing nodemailer/SendGrid setup in functions/index.js
- Subject: [Support Escalated] {tenantName} — {userRole}: {first 60 chars of
message}
- Body: tenant name, user name, user role, full message, account created date,
recent booking statuses, link to admin support queue

Export handleSupportMessage from functions/index.ts.
Write tests for the trigger guard (role != "user" returns early) and the
escalation write.
```

SUPPORT TASK 4 — AdminSupportQueueScreen (run in Week 6)

```
Create src/screens/admin/AdminSupportQueueScreen.tsx. This screen is visible only
to the platform owner role.

Requirements:
- Subscribe to escalationQueue collection where status == "pending", orderBy
createdAt desc
- List view: each row shows tenant name, user display name, user role badge
(client/admin), first 80 chars of message, createdAt relative time, AI confidence
badge (if set)
- Tap row → open ticket detail view (either modal or pushed screen)
- Ticket detail view shows:
  - Full user context: tenant, name, role, account age, recent booking statuses
  - Full message thread from tenants/{tenantId}/supportTickets/{ticketId}/messages
  - AI reason field (why it was escalated)
  - Reply text input pre-populated with draftReply (if set)
  - Three action buttons: Send Reply, Dismiss (no reply), Re-route to AI (reserved
for Phase 2, disabled for now)
- Send Reply: writes a new message document (role: "admin", senderId: auth.uid) to
the messages subcollection, updates ticket status to "resolved", updates
escalationQueue document status to "replied"
- Dismiss: updates ticket status to "closed", escalationQueue status to
"dismissed"
- Empty state when queue is empty: "All clear — no pending support requests"
- Add screen to admin navigation stack (owner role only, not visible to tenant
admins)
```

SUPPORT TASK 5 — AI Integration (run in Week 9)

```
Upgrade the handleSupportMessage Cloud Function in functions/src/support.ts to
activate AI routing. This is Phase 2.

Requirements:
1. Install openai npm package in functions/
2. Load OpenAI API key from Firebase Functions config:
functions.config().openai.key
3. Load platform/config document from Firestore to get: supportSystemPrompt,
supportEscalationThreshold (default 0.75), supportEscalationEmail
4. Replace the Phase 1 auto-escalate stub with a real OpenAI GPT-4o call:
   - Use chat completions with response_format: { type: "json_schema" }
   - Schema: { answer: string, confidence: number (0–1), escalate: boolean,
reason: string }
   - System prompt = supportSystemPrompt + context injection (user role, tenant
name, recent booking statuses, locale, account age)
   - User message = message.content
5. Parse response:
   - If escalate == false AND confidence >= supportEscalationThreshold:
       → Write AI answer as new message (role: "ai") in messages subcollection
       → Set ticket status = "resolved", aiConfidence, aiReason
       → Do NOT write to escalationQueue
   - Else:
       → Set ticket status = "escalated", draftReply = answer, aiConfidence,
aiReason
       → Write to escalationQueue with draftReply populated
       → Send escalation email (now includes AI draft in body)
6. Hard-coded always-escalate check BEFORE the AI call: if message content matches
any of these patterns, set escalate=true immediately without calling the AI:
billing|payment|refund|delete my account|gdpr|data request|harassment|abuse
7. Add try/catch around the OpenAI call: on error, fall back to escalation (never
fail silently to the user)
8. Update the "Re-route to AI" button in AdminSupportQueueScreen to call a new
Cloud Function retryAISupportTicket that re-runs the AI on the original message
with the current knowledge base.
```

SUPPORT TASK 6 — Polish and Tuning (run in Weeks 11–12)

```
Add final support system polish:

1. Auto-close scheduled function:
   - Cloud Function on a daily schedule
   - Query all supportTickets across tenants where status == "resolved" and
updatedAt < now - 48h
   - Update status to "closed"
   - This requires a collection group query on supportTickets — add the necessary
Firestore composite index
```

```
2. Knowledge base editor in AdminSupportQueueScreen (owner only):
   - Add a "Knowledge Base" tab or modal
   - TextInput (multiline, monospace font) bound to
platform/config.supportSystemPrompt
   - Save button, confirmation toast
   - Threshold input: numeric field bound to
platform/config.supportEscalationThreshold (0.0–1.0)

3. Support analytics card in admin dashboard:
   - Total tickets this month
   - AI resolution rate (% resolved without escalation)
   - Average AI confidence on resolved tickets
   - Average response time on escalated tickets
   - Query all three from a collection group query on supportTickets and
escalationQueue

4. Rate limiting:
   - In handleSupportMessage: count open tickets for userId in past 24h
   - If count >= 10, reject with a polite message: "Too many open requests. Please
wait for a response before opening new ones."
   - Do not write to escalationQueue for rate-limited requests

5. AI message badge:
   - In SupportChatScreen, messages with role == "ai" show a small "✦ AI" badge
   - Add a discreet footer: "Initial responses are handled by AI. If you need
human support, your request will be escalated."
```

---

12. Open Decisions Register

Track decisions yet to be made here. Update this section as you decide.

#	Decision	Options	Status	Notes
1	OpenAI model version	gpt-4o vs gpt-4o-mini	Open	gpt-4o-mini is 10× cheaper; try mini first, upgrade if quality insufficient
2	Knowledge base storage	Firestore field vs Storage file	Decided: Firestore field	Simpler; can revisit if prompt exceeds 10k chars
3	RAG upgrade trigger	Never / >500 tickets/month / if AI accuracy < 70%	Open	Revisit at 3 months of production data
4	Multi-language support	English only / per-tenant locale / auto-detect	Open	GPT-4o handles this naturally; set response language in system prompt to match user's locale field
5	Proactive support	None / trigger after booking failure / trigger after repeated app errors	Open	Defer to Phase 3; requires error telemetry first

#	Decision	Options	Status	Notes
6	User rating of AI replies	Thumbs up/down / none	Open	Simple to add; useful for tuning; consider in Phase 3

End of document. Cross-reference `MULTITENANT_MASTER_INDEX.md` for the full program map.